The Marathon 2: A Navigation System

Steve Macenski
R&D Innovations
Samsung Research
s.macenski@samsung.com

Francisco Martín
Intelligent Robotics Lab
Rey Juan Carlos University
francisco.rico@urjc.es

Ruffin White
Contextual Robotics Institute
UC San Diego
rwhitema@eng.ucsd.edu

Jonatan Gins Clavero
Intelligent Robotics Lab
Rey Juan Carlos University
jonatan.gines@urjc.es

***Abstract*—Developments in mobile robot navigation have enabled robots to operate in warehouses, retail stores, and on sidewalks around pedestrians. Various navigation solutions have been proposed, though few as widely adopted as ROS (Robot Operating System) Navigation. 10 years on, it is still one of the most popular navigation solutions[1]. Yet, ROS Navigation has failed to keep up with modern trends. We propose the new navigation solution, *Navigation2*, which builds on the successful legacy of ROS Navigation. Navigation2 uses a behavior tree for navigator task orchestration and employs new methods designed for dynamic environments applicable to a wider variety of modern sensors. It is built on top of ROS2, a secure message passing framework suitable for safety critical applications and program lifecycle management. We present experiments in a campus setting utilizing Navigation2 to operate safely alongside students over a marathon as an extension of the experiment proposed in Eppstein et al. [1]. The Navigation2 system is freely available at https://github.com/ros-planning/navigation2 with a rich community and instructions.**

***Index Terms*—Service Robots; Software, Middleware and Programming Environments; Behaviour-Based Systems**

I. Introduction

Many mobile robot navigation frameworks and systems have been proposed since the first service robots were created [2] [3]. These frameworks laid the groundwork for the service robots that are being rolled out in factories, retail stores, and on sidewalks today. While many have been proposed, very few have rivaled the impact of the *ROS Navigation Stack* on the growing mobile robotics industry.

Indeed, this open-source project has fueled companies, governments, and researchers alike. It has gathered more than 400 citations, over 1,000 papers indexed by Google Scholar mention it, and 1,200 forks on GitHub, as of February 2020. Proposed in 2010, it provided a rich and configurable navigation system that has been reconfigured for use on a variety of robot platforms [1]. It provided a reference framework and a set of foundational implementations of algorithms including A* [4] and Dynamic Window Approach (DWA) [5] for planning and control. Navigation, however, suffered from perception and controller issues in highly dynamic environments and lacked a general state estimator. Further, ROS1 lacks in security and performance aspects, rendering it unsuitable for safety critical applications.

[1]By GitHub stars, forks, citations, and industry use

(a) Tiago [12]   (b) RB-1 [13]

Fig. 1: Robots used for the marathon experiments.

In this work, we studied current trends and robotics systems to create a new navigation system leveraging our collective experiences working with popular robotics frameworks. This work looks to build off of the success of Navigation while supporting a wider variety of robot shapes and locomotion types in more complex environments.

In this paper, we propose a new, fully open-source, navigation system with substantial structural and algorithmic refreshes, *Navigation2*. Navigation2 uses a configurable behavior tree to orchestrate the planning, control, and recovery tasks [6]. Its federated model is structured such that each behavior tree node invokes a remote server to compute one of the above tasks with one of a growing number of algorithm implementations. Each server implements a standard plugin interface to allow for new algorithms or techniques to be easily created and selected at run-time.

This architecture makes use of multi-core processors and leverages the real-time, low-latency capabilities of ROS2 [7]. ROS2 was redesigned from the ground up to be industrial-grade providing security, reliability, and real-time capabilities

马拉松 2：导航系统

史蒂夫·马肯斯基 *R&D Innovations* s.macenski@samsung.com *Samsung Research*
弗朗西斯科·马丁·里科 *Intelligent Robotics Lab* francisco.rico@urjc.es *Rey Juan Carlos University*
鲁芬·怀特 *Contextual Robotics Institute* rwhitema@eng.ucsd.edu *UC San Diego*
乔纳坦·金斯·克拉韦罗 *Intelligent Robotics Lab* jonatan.gines@urjc.es *Rey Juan Carlos University*

Abstract*—**移动机器人导航的发展使机器人能够在仓库、零售店和行人周围的人行道上运行。人们已经提出了各种导航解决方案，但很少有像 ROS（机器人操作系统）导航那样广泛采用的解决方案。 10 年过去了，它仍然是最受欢迎的导航解决方案之一[1]。然而，ROS 导航未能跟上现代趋势。我们提出了新的导航解决方案 ***Navigation2，它建立在 ROS 导航的成功基础上。 Navigation2 使用行为树进行导航器任务编排，并采用专为动态环境设计的新方法，适用于更广泛的现代传感器。它建立在 ROS2 之上，ROS2 是一个安全消息传递框架，适用于安全关键应用程序和程序生命周期管理。我们在校园环境中进行了实验，利用 Navigation2 与学生一起安全地参加马拉松比赛，作为 Eppstein 等人提出的实验的延伸。 [1]。 Navigation2 系统可在 https://github.com/ros-planning/navigation2 免费获取，拥有丰富的社区和说明。

***Index Terms*——**服务机器人；软件、中间件和编程环境；基于行为的系统

一、简介

自从第一个服务机器人诞生以来，许多移动机器人导航框架和系统已经被提出[2] [3]。这些框架为如今在工厂、零售店和人行道上推出的服务机器人奠定了基础。尽管已经提出了许多建议，但很少有人能与 *ROS Navigation Stack* 对不断发展的移动机器人行业的影响相媲美。

事实上，这个开源项目已经推动了公司、政府和研究人员的发展。截至 2020 年 2 月，它已被引用超过 400 次，超过 1,000 篇论文被 Google Scholar 索引，在 GitHub 上有 1,200 个分支。它于 2010 年提出，提供了一个丰富且可配置的导航系统，该系统经过重新配置，可在各种机器人平台上使用 [1]。它提供了一个参考框架和一组算法的基础实现，包括用于规划和控制的 A* [4] 和动态窗口方法 (DWA) [5]。然而，导航在高度动态的环境中受到感知和控制器问题的影响，并且缺乏通用的状态估计器。此外，ROS1 在安全性和性能方面缺乏，使其不适合安全关键应用。

1按 GitHub 星标、分叉、引用和行业使用

(a) Tiago [12] (b) RB-1 [13]

图1：马拉松实验所用机器人 恩特。

在这项工作中，我们研究了当前的趋势和机器人系统，利用我们与流行机器人框架合作的集体经验来创建一个新的导航系统。这项工作旨在以导航的成功为基础，同时在更复杂的环境中支持更广泛的机器人形状和运动类型。

在本文中，我们提出了一种新的、完全开源的导航系统，该系统在结构和算法上进行了大量更新，*Navigation2*。Navigation2 使用可配置的行为树来编排规划、控制和恢复任务[6]。其联合模型的结构使得每个行为树节点调用远程服务器，以使用越来越多的算法实现之一来计算上述任务之一。每个服务器都实现一个标准插件接口，以允许在运行时轻松创建和选择新算法或技术。

该架构利用多核处理器并利用 ROS2 [7] 的实时、低延迟功能。 ROS2 从头开始重新设计为工业级，提供安全性、可靠性和实时功能

for the next generation of robots. ROS2 also introduces the concept of *Managed Nodes*, servers whose life-cycle state can be controlled, which we exploit to create deterministic behavior for each server in the system.

Our proposed algorithmic refreshes are focused on modularity and operating smoothly in dynamic environments. This includes: *Spatio-Temporal Voxel Layer* (STVL) [8], layered costmaps [9], *Timed Elastic Band* (TEB) controller [10], and a multi-sensor fusion framework for state estimation, *Robot Localization* [11]. Each supports holonomic and non-holonomic robot types. However, Navigation2 is specifically designed such that *all* components are run-time configurable and in many cases, multiple alternative algorithms are already available for use. Our navigation system, the first of its kind built using ROS2, is shown to safely operate in a highly-dynamic campus setting. Through long-duration experiments, we show that it can robustly navigate a distance in excess of a marathon (37.4 miles total) with the Tiago & RB-1 base, shown in Figure 1, through high-trafficked areas.

II. Related Work

A. Navigation Systems

Dervish was one of the first robots that implemented an effective navigation system in an office environment [14]. It used a coarse probabilistic representation of the environment using a sonar ring to navigate through a hallway scene for the Office Delivery Robot Competition. Soon after, [2] and [15] introduced the museum robots, RHINO and MINERVA. These robots primarily used probabilistic occupancy grids to localize in their environments with varying planning approaches. To avoid obstacles, DWA was used [5]. These robots relied on 2D environmental representations for positioning and navigation, which could lead to collisions with out of plane obstacles.

[1] presented a navigation system and environmental representation utilizing 3D information from tilting 2D laser scanners to navigate an environment. Many of the early techniques, such as A* and DWA, were used for planning and obstacle avoidance. Over time, tilting 2D laser scanners have given way to ubiquitous inexpensive 3D depth cameras and laser scanners. Some of the approach's formulation is specialized for the geometry, low accuracy, and effects displayed by a 2D tilting laser configuration. Additionally, with the introduction of sparse multi-beam laser scanners, pure raycasting methods can no longer effectively clear out free space, especially in a dynamic scene. STVL created a more scalable alternative for many, sparse, and long-range sensors that is utilized in place of such techniques [8].

An outdoor robot navigation experiment was presented in 2013 [19]. That work focused on the mapping, localization, and simple dynamic obstacle detection method used to navigate the streets of Germany over 7 km. Our work focuses on a modular navigation framework to build many types of applications using behavior trees and extended experimentation to showcase its maturity. While we do not directly consider the detection of dynamic obstacles in our experiments, all the necessary interfaces exist to do this through Navigation2 with an use-case specific detector enabled. We implemented an optimal local trajectory planner and 3D environmental modelling technique that is functionally efficient in the presence of dynamic obstacles without explicit detection, but can be augmented with such information if available.

B. Models

Behavior trees (BT) have been shown to be successful for task planning in robotics, as a replacement for finite state machines (FSM) [20]. With growing numbers of states and transitions, modeling complex behaviors and multi-step tasks as FSMs can become intractably complex for real-world tasks. [20] successfully demonstrates the use of behavior trees to model motion and shooting tactics for soccer game play that would otherwise be intractable with a FSM.

BTs have also been applied heavily to the domain of robot mission execution, robot manipulation, and complex mobile-manipulator tasks [6]. Further, behavior trees can be easily modified with widely reusable primitives and loaded at run-time, requiring no programmed logic. For this reason, behavior trees are used in our approach as the primary structure of the navigation framework.

III. Navigation2 Design

The Navigation2 system was designed for a high-degree of configurability and future expansion. While the marathon experiments display one such configuration of Navigation2, the intent of our work is to support a large variety of robot types (differential, holonomic, ackermann, legged) for a large variety of environments and applications. Further, this support is not only targeted for research and education, but for production robots as well. The design took into account the requirements for robotics products needs including safety, security, and determinism without loss of the above generality.

A. Reliable

For professional mobile robots moving in excess of 2 m/s near humans, there are many safety and reliability concerns that must be addressed. Navigation2 is designed on top of a real-time capable meta-operating system, ROS2, to address functional safety standards and determinism. ROS2 is the second generation of ROS, the popular robot middleware, built on Data Distribution Service (DDS) communication standard [7]. DDS is used in critical infrastructure including aircraft, missile systems, and financial systems. It exposes the same strict messaging guarantees from the DDS standard to enable applications to select a policy for messaging. ROS2 leverages DDS security features to allow a user to securely transmit information inside the robot and to cloud servers without a dedicated network. These features create a viable communication framework for industrial-grade mobile robots.

Further, ROS2 introduces the concept of Managed Nodes (also known as Lifecycle Nodes). A managed node implements a server structure with clear state transitions from instantiation through destruction [21]. This server is created when the program is launched, but waits for external stimulus to transition

用于下一代机器人。 ROS2 还引入了 *Managed Nodes* 的概念，即生命周期状态可以控制的服务器，我们利用它为系统中的每个服务器创建确定性行为。

我们提出的算法更新侧重于模块化和动态环境中的平稳运行。这包括：*Spatio-Temporal Voxel Layer* (STVL) [8]、分层代价图 [9]、*Timed Elastic Band* (TEB) 控制器 [10] 和用于状态估计的多传感器融合框架 *Robot Localization* [11]。每个都支持完整和非完整机器人类型。然而，Navigation2 经过专门设计，使得 *all* 组件在运行时可配置，并且在许多情况下，已有多种替代算法可供使用。我们的导航系统是同类中第一个使用 ROS2 构建的导航系统，经证明可以在高度动态的校园环境中安全运行。通过长时间的实验，我们表明，它可以使用 Tiago 和 RB-1 底座稳健地导航超过马拉松的距离（总共 37.4 英里），如图 1 所示，穿过人流量大的区域。

二.相关工作

A. *Navigation Systems*

Dervish 是第一批在办公环境中实施有效导航系统的机器人之一 [14]。它使用声纳环对环境进行粗略概率表示，在办公室交付机器人竞赛的走廊场景中导航。不久之后，[2]和[15]介绍了博物馆机器人RHINO和MINERVA。这些机器人主要使用概率占用网格通过不同的规划方法在其环境中进行定位。为了避开障碍物，使用了 DWA [5]。这些机器人依靠二维环境表示进行定位和导航，这可能会导致与平面外障碍物发生碰撞。

[1] 提出了一种导航系统和环境表示，利用来自倾斜 2D 激光扫描仪的 3D 信息来导航环境。许多早期技术，例如 A* 和 DWA，用于规划和避障。随着时间的推移，倾斜 2D 激光扫描仪已经让位于无处不在的廉价 3D 深度相机和激光扫描仪。该方法的一些公式专门针对几何形状、低精度以及二维倾斜激光配置显示的效果。此外，随着稀疏多光束激光扫描仪的引入，纯光线投射方法不再能够有效地清除自由空间，尤其是在动态场景中。 STVL 为许多稀疏的远程传感器创建了一种更具可扩展性的替代方案，用于代替此类技术 [8]。

2013 年提出了户外机器人导航实验[19]。这项工作的重点是地图绘制、定位和简单的动态障碍物检测方法，用于在德国街道上行驶 7 公里以上。我们的工作重点是模块化导航框架，使用行为树和扩展实验来构建多种类型的应用程序以展示其成熟度。虽然我们在实验中没有直接考虑动态障碍物的检测，但通过 Navigation2 存在所有必要的接口来做到这一点启用特定于用例的检测器。我们实现了最佳局部轨迹规划器和 3D 环境建模技术，该技术在存在动态障碍物且无需明确检测的情况下功能高效，但如果可用，可以使用此类信息进行增强。

B. *Models*

行为树（BT）已被证明可以成功用于机器人任务规划，作为有限状态机（FSM）的替代品[20]。随着状态和转换数量的不断增加，将复杂行为和多步骤任务建模为 FSM 对于现实世界的任务来说可能会变得极其复杂。 [20] 成功演示了使用行为树来模拟足球比赛的运动和射门策略，否则 FSM 很难处理。

BT 也被大量应用于机器人任务执行、机器人操纵和复杂的移动操纵器任务领域 [6]。此外，行为树可以使用广泛可重用的原语轻松修改，并在运行时加载，不需要编程逻辑。因此，我们的方法中使用行为树作为导航框架的主要结构。

三．导航2设计

Navigation2 系统专为高度可配置性和未来扩展而设计。虽然马拉松实验展示了一种这样的 Navigation2 配置，但我们工作的目的是支持多种机器人类型（差分机器人、完整机器人、阿克曼机器人、腿式机器人），以适应各种环境和应用。此外，这种支持不仅针对研究和教育，还针对生产机器人。该设计考虑了机器人产品需求的要求，包括安全性、保密性和确定性，同时又不失上述通用性。

A. *Reliable*

对于在人类附近移动速度超过 2 m/s 的专业移动机器人，必须解决许多安全性和可靠性问题。 Navigation2 是在实时元操作系统 ROS2 之上设计的，旨在解决功能安全标准和确定性问题。 ROS2 是第二代 ROS，是流行的机器人中间件，基于数据分发服务（DDS）通信标准构建[7]。 DDS 用于关键基础设施，包括飞机、导弹系统和金融系统。它公开了与 DDS 标准相同的严格消息传递保证，使应用程序能够选择消息传递策略。 ROS2 利用 DDS 安全功能，允许用户在机器人内部安全地传输信息并传输到云服务器，而无需专用网络。这些功能为工业级移动机器人创建了可行的通信框架。此外，ROS2引入了受管节点（也称为生命周期节点）的概念。受管节点实现了一个服务器结构，该结构具有从实例化到销毁的清晰状态转换[21]。该服务器在程序启动时创建，但等待外部刺激进行转换

through a deterministic bringup process. At shutdown or error, the server is stepped through its state machine from an active to finalized state. Each state transition has clear responsibilities including management of memory allocation, networking interfaces, and beginning to process its task. All servers in Navigation2 make use of managed nodes for deterministic program lifecycle management and memory allocations.

B. Modular and Reconfigurable

To create a navigation system that can work with a large variety of robots in many environments, Navigation2 must be highly modular. Similarly, our design approach favored solutions that were not simply modular, but also easy to reconfigure and select at run-time. Navigation2 created two design patterns to accomplish this: a behavior tree navigator and task-specific asynchronous servers, shown in Figure 2. Each task-specific server is a ROS2 node hosting algorithm plugins, which are libraries dynamically loaded at run-time.

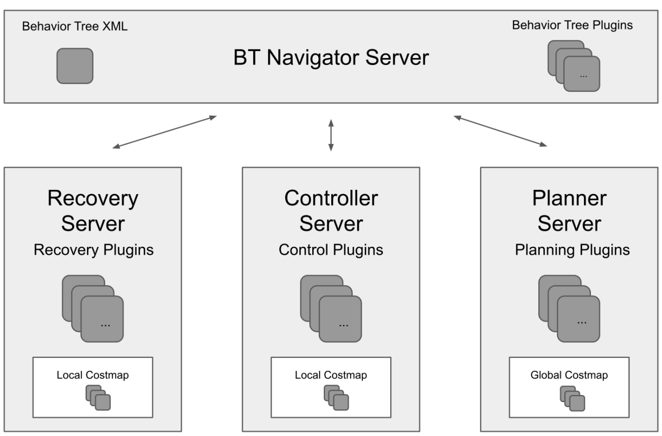

Fig. 2: Overview of Navigation2 design.

The *Behavior Tree Navigator* uses a behavior tree to orchestrate the navigation tasks; it activates and tracks progress of planner, controller, and recovery servers to navigate. The behavior tree plugins call to the planner, controller and recovery servers using the ROS2 action interface. Unique navigation behaviors can be created by modifying a behavior tree, stored as an XML. BTs are trivial to reconfigure with different control flow and condition node types, requiring no programming. Changing navigation logic is as simple as creating a new behavior tree markup file that is loaded at run-time. We exploit BT action node statuses and control flow nodes to create a contextual recovery system where the failure of a specific server will trigger a unique response.

Using ROS2, the behavior tree nodes in the navigator can call long running asynchronous servers in other processor cores. Making use of multi-core processors substantially increases the amount of compute resources a navigation system can effectively utilize. The Navigation2 BT navigator makes use of dynamic libraries to load plugins of BT nodes. This pattern allows for reusable primitive nodes to be created and loaded with a behavior tree XML at run-time without linking to the navigator itself. More than simply utilizing multiple processor cores, these nodes can call remote servers on other CPUs in any language with ROS2 client library support.

The task-specific servers are designed to each host a ROS2 server, environmental model, and run-time selected algorithm plugin. They are modular such that none or many of them can run at the same time to compute actions. The ROS2 server is the entry-point for BT navigator nodes. This server also processes cancellation, preemption, or new information requests. The requests are forwarded to the algorithm plugin to complete their task with access to the environmental model.

In summary, we can create configurable behavior trees whose structure and nodes can be loaded at run-time. Each BT node manages the communications with a remote server, that hosts an algorithm written in an array of languages, loaded at run-time through plugin interfaces.

C. Support Feature Extensions

Using the modularity and configurability of Navigation2, several commercial feature extensions were proposed in the design phase. This subsection will highlight key examples and the design considerations to enable them. Behavior trees are most frequently used to model complex or multi-step tasks, where navigating to a position is a node of that larger task. The BT library *BehaviorTree.CPP*[2] was selected for its popularity and support for subtrees to allow Navigation2 to be used in larger BT tasks. Users with complex missions may use a provided 'Navigation2 BT node' wrapper to use Navigation2 as a subtree of their mission.

Many service robots will autonomously dock to recharge or operate elevators without requiring human assistance [22]. Traditional navigation frameworks have not addressed this class of task in their formulation. While these extensions are not implemented due to lack of vendor standardization, we provide instruction to easily enable them. Each server supports multiple algorithm plugins; a docking controller algorithm can be loaded into the controller server and called from the BT to dock a robot. A BT action node can be trivially added to call an elevator using other IoT APIs provided by vendors [23].

IV. Navigation2 Implementation

In this section, each of the recovery, planner, and controller servers provided in Navigation2 are examined. Each may be customized or replaced by the user, but we will address the specific algorithm plugins and servers provided, recommended, and used in our experiment. Each server contains an environmental model relevant for their operation, a network interface to ROS2, and a set of algorithm plugins to be defined at run-time. Provided plugins can be seen in Table I.

A. Behavior Tree Navigator

The behavior tree navigator is the highest level component of Navigation2, hosting the tree used to implement navigation behaviors. Navigation2 builds upon BehaviorTree.CPP, an open source C++ library supporting type-safe asynchronous

[2]https://github.com/BehaviorTree/BehaviorTree.CPP

通过确定性的启动过程。在关闭或出错时，服务器会通过其状态机从活动状态逐步进入最终状态。每个状态转换都有明确的职责，包括管理内存分配、网络接口以及开始处理其任务。 Navigation2 中的所有服务器都利用托管节点来进行确定性程序生命周期管理和内存分配。

B. Modular and Reconfigurable

为了创建一个可以在多种环境下与多种机器人配合使用的导航系统，Navigation2 必须高度模块化。同样，我们的设计方法青睐的解决方案不仅是简单的模块化，而且还易于在运行时重新配置和选择。 Navigation2 创建了两种设计模式来实现此目的：行为树导航器和特定于任务的异步服务器，如图 2 所示。每个特定于任务的服务器都是一个托管算法插件的 ROS2 节点，这些插件是在运行时动态加载的库。

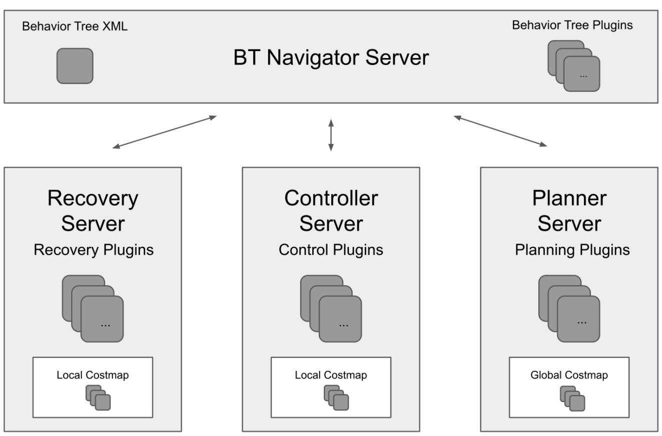

图 2：Navigation2 设计概述。

Behavior Tree Navigator 使用行为树来编排导航任务；它激活并跟踪规划器、控制器和恢复服务器的进度以进行导航。行为树插件使用 ROS2 操作接口调用规划器、控制器和恢复服务器。可以通过修改以 XML 形式存储的行为树来创建独特的导航行为。 BT 可以轻松地重新配置不同的控制流和条件节点类型，无需编程。更改导航逻辑就像创建在运行时加载的新行为树标记文件一样简单。我们利用 BT 操作节点状态和控制流节点来创建一个上下文恢复系统，其中特定服务器的故障将触发独特的响应。

使用ROS2，导航器中的行为树节点可以调用其他处理器内核中长时间运行的异步服务器。利用多核处理器大大增加了导航系统可以有效利用的计算资源量。 Navigation2 BT导航器利用动态库来加载BT节点的插件。该模式允许在运行时创建可重用的原始节点并使用行为树 XML 加载，而无需链接到导航器本身。这些节点不仅仅是利用多个处理器核心，还可以通过 ROS2 客户端库支持以任何语言调用其他 CPU 上的远程服务器。

特定于任务的服务器被设计为每个主机都有一个 ROS2 服务器、环境模型和运行时选择的算法插件。它们是模块化的，因此它们中的任何一个或多个都不能同时运行来计算操作。 ROS2 服务器是 BT 导航器节点的入口点。该服务器还处理取消、抢占或新信息请求。请求被转发到算法插件以完成其任务并访问环境模型。

总之，我们可以创建可配置的行为树，其结构和节点可以在运行时加载。每个 BT 节点管理与远程服务器的通信，该服务器托管用多种语言编写的算法，并通过插件接口在运行时加载。

C. Support Feature Extensions

利用 Navigation2 的模块化和可配置性，在设计阶段提出了几个商业功能扩展。本小节将重点介绍关键示例以及实现这些示例的设计注意事项。行为树最常用于对复杂或多步骤任务进行建模，其中导航到某个位置是该较大任务的一个节点。 BT 库 *BehaviorTree.CPP*[2] 因其流行性和对子树的支持而被选中，以允许 Navigation2 在更大的 BT 任务中使用。具有复杂任务的用户可以使用提供的“Navigation2 BT 节点”包装器来使用 Navigation2 作为其任务的子树。

许多服务机器人将自动对接充电或操作电梯，无需人工协助[22]。传统的导航框架在其制定中并未解决此类任务。虽然由于缺乏供应商标准化而未实现这些扩展，但我们提供了轻松启用它们的说明。每个服务器支持多种算法插件；对接控制器算法可以加载到控制器服务器中并从 BT 调用来对接机器人。可以简单地添加 BT 操作节点，以使用供应商提供的其他 IoT API 来呼叫电梯 [23]。

四．导航2实施

在本节中，将检查 Navigation2 中提供的每个恢复、规划器和控制器服务器。每个都可以由用户定制或替换，但我们将解决在我们的实验中提供、推荐和使用的特定算法插件和服务器。每个服务器都包含与其操作相关的环境模型、ROS2 的网络接口以及一组在运行时定义的算法插件。提供的插件见表一。

A. Behavior Tree Navigator

行为树导航器是 Navigation2 的最高级别组件，托管用于实现导航行为的树。 Navigation2 基于BehaviorTree.CPP 构建，这是一个支持类型安全异步的开源 C++ 库

2https://github.com/BehaviorTree/BehaviorTree.CPP

TABLE I: Navigation2 provided plugins.

Type	Plugin	Description
Control	DWB Controller	Configurable DWA controller
	TEB Controller	Timed-Elastic-Bands controller
Costmap	**Inflation Layer**	Inflate obstacles in costmap
	Non-Persist. Voxel	Maintain only recent voxels
	Obstacle Layer	Raycast 2D obstacles
	STVL	Temporal 3D sparse voxel grid
	Static Layer	Loads static map into costmap
	Voxel Layer	Raycast 3D obstacles
Planner	**NavFn Planner**	Holonomic A* expansion
Recovery	Back Up	Back out of sticky situations
	Clear Costmap	Clear erroneous measurements
	Spin	Rotate to clear free space
	Wait	Waitout time based obstacles

Bold entries used in experiments.

actions, composable trees, and logging/profiling infrastructures for development. The top level node, BT navigator server, seen in Figure 2, is where user defined BT are loaded and run. Each node calls a server, below, to complete a task.

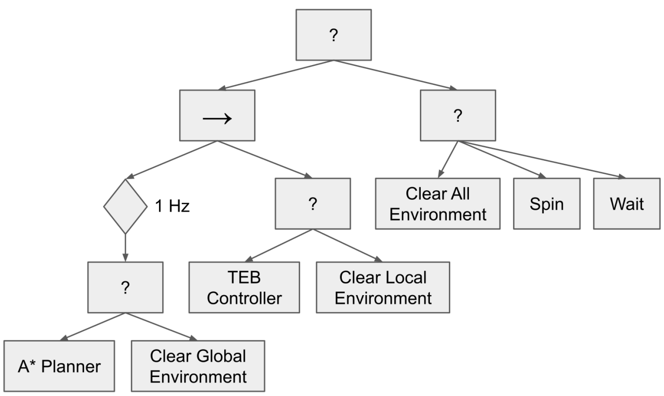

Fig. 3: Behavior tree used in the marathon. '?' represent a fallback node, and "→" represents a sequence node.

Figure 3 depicts a conventional Navigation2 BT, also used in our experiments. Following the logical flow from left to right, a policy ticks the global planner action at a rate of 1 Hz. If the global planner fails to find a path, the fallback node advances to the next child action to clear the global environment of prior obstacles. Clearing of the environmental model may help resolve potential failures in the perception system. The parent sequence node then ticks the controller or similar clear fallback behavior. Should local fallback actions and subsequent sequence node return failure, the root fallback node then ticks the final child, attempting evermore aggressive recovery behaviors. First by cleaning all environments, then spinning the robot in place to reorient local obstacles, and finally waiting for any dynamic obstacles to give way.

B. Recovery

Recovery behaviors are used to mitigate complete navigation failures. They are called from the leaves in the behavior tree and carried out by the recovery server. By convention, these behaviors are ordered from conservative to aggressive actions.

Note, however, that recovery behaviors can either be specific to its subtree (e.g. the global planner or controller) or system level in the subtree containing only recoveries in case of system failures. Those used for experiments are as follows:

Clear Costmap: A recovery to clear costmap layers in case of perception system failure. **Spin**: A recovery to clear out free space and nudge robot out of potential local failures, e.g. the robot perceives itself to be too entrapped to back out. **Wait**: A recovery to wait in case of time-based obstacle like human traffic or collecting more sensor data.

C. Perception

As range and resolution of depth sensing technologies have improved, so too have world representations for modeling a robot's environment. To leverage these improvements, a layered costmap approach is used, allowing the user to customize the hierarchy of layers from various sensor modalities, resolutions, and rate limits. A layered costmap allows for a single costmap to be coherently updated by a number of data sources and algorithms. Each layer can extend or modify the costmap it inherits, then forward the new information to planners and controllers.

While the costmap is two dimensional, STVL maintains a 3D representation of the environment to project obstacles into the planning space. This layer uses temporal-based measurement persistence to maintain an accurate view of the world in the presence of dynamic obstacles. It scales better than raycasting approaches with many, high-resolution, and long-range sensors such as 2D and 3D laser scanners, depth cameras, and Radars [8]. Figure 4 shows the robot planning a route in a central stairwell and its view of the environment using STVL to represent a voxel grid. The map (shown in black) and inflation of the static map (colored gradient) are also displayed in the local and global costmaps.

Those used for experiments are as follows:

Static Layer: Uses the static map, provided by a SLAM pipeline or loaded from disk, and initializes occupancy information. **Inflation Layer**: Inflates lethal obstacles in costmap with exponential decay by convolving the collision footprint of the robot. **Spatio-Temporal Voxel Layer**: Maintains temporal 3D sparse volumetric voxel grid that decays over time via sensor models from the laser and RGBD cameras.

D. Global Planner and Controller

The task of the global planner is to compute the shortest route to a goal and the controller uses local information to compute the best local path and control signals. The global planner and controller are plugins delegated to task-specific asynchronous servers. For performance, relevant global and local costmaps are co-located in their respective server.

New algorithms are employed for smoothly navigating in highly dynamic environments by combining the TEB Local Planners with the STVL for dynamic 3D perception. The TEB controller is capable of taking in object detections and tracks to inform the controller of additional constraints on the environment. It is also suitable for use on differential, omnidirectional,

表 I：Navigation2 提供的插件。

Type	Plugin	Description
Control	DWB Controller	Configurable DWA controller
	TEB Controller	Timed-Elastic-Bands controller
Costmap	**Inflation Layer**	Inflate obstacles in costmap
	Non-Persist. Voxel	Maintain only recent voxels
	Obstacle Layer	Raycast 2D obstacles
	STVL	Temporal 3D sparse voxel grid
	Static Layer	Loads static map into costmap
	Voxel Layer	Raycast 3D obstacles
Planner	**NavFn Planner**	Holonomic A* expansion
Recovery	Back Up	Back out of sticky situations
	Clear Costmap	Clear erroneous measurements
	Spin	Rotate to clear free space
	Wait	Waitout time based obstacles

实验中使用的粗体条目。

用于开发的操作、可组合树和日志记录/分析基础设施。顶层节点，BT 导航服务器，如图 2 所示，是用户定义的 BT 加载和运行的地方。每个节点调用下面的服务器来完成任务。

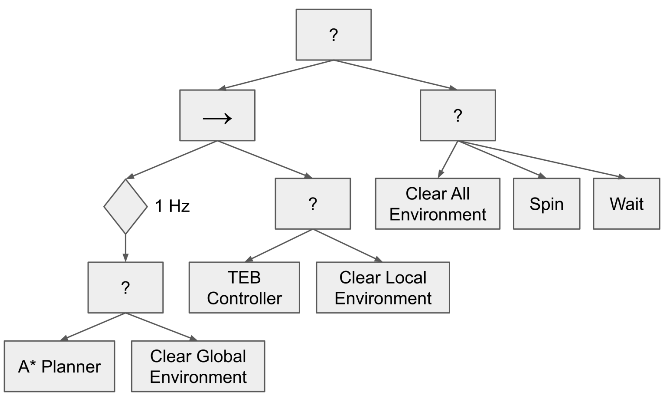

图3：马拉松比赛中使用的行为树。 '?' 代表后备节点，“→” 代表序列节点。

图 3 描绘了我们的实验中也使用的传统 Navigation2 BT 。遵循从左到右的逻辑流程，策略以 1 Hz 的速率勾选全局规划器操作。如果全局规划器未能找到路径，后备节点将前进到下一个子操作，以清除全局环境中先前的障碍。环境模型的清除可能有助于解决感知系统中潜在的故障。然后，父序列节点会勾选控制器或类似的明确回退行为。如果本地回退操作和后续序列节点返回失败，则根回退节点会勾选最后一个子节点，尝试更加积极的恢复行为。首先清洁所有环境，然后将机器人旋转到位以重新调整局部障碍物的方向，最后等待任何动态障碍物让路。

B. Recovery

恢复行为用于缓解完全导航失败的情况。它们从行为树的叶子中调用并由恢复服务器执行。按照惯例，这些行为按照从保守到激进的顺序排列。

但请注意，恢复行为可以特定于其子树（例如全局规划器或控制器），也可以特定于子树中仅包含系统故障情况下的恢复的系统级别。实验所用的如下：

清除成本图：在感知系统故障的情况下恢复清除成本图层。自旋：清除可用空间并推动机器人摆脱潜在局部故障的恢复，例如机器人认为自己被困得太深而无法退出。等待：在出现基于时间的障碍（例如人员交通或收集更多传感器数据）时等待的恢复。

C. Perception

随着深度传感技术的范围和分辨率不断提高，机器人环境建模的世界表征也不断提高。为了利用这些改进，使用了分层成本图方法，允许用户根据各种传感器模式、分辨率和速率限制自定义层的层次结构。分层成本图允许多个数据源和算法连贯地更新单个成本图。每层都可以扩展或修改它继承的代价图，然后将新信息转发给规划器和控制器。

虽然成本图是二维的，但 STVL 保留了环境的 3D 表示，以将障碍物投影到规划空间中。该层使用基于时间的测量持久性来在存在动态障碍的情况下保持准确的世界视图。它比使用许多高分辨率和长距离传感器（例如 2D 和 3D 激光扫描仪、深度相机和雷达）的光线投射方法具有更好的扩展性[8]。图 4 显示了机器人在中央楼梯间规划路线及其使用 STVL 表示体素网格的环境视图。地图（以黑色显示）和静态地图的膨胀（彩色渐变）也显示在局部和全局成本图中。

实验所用的如下：
静态层：使用由 SLAM 管道提供或从磁盘加载的静态地图，并初始化占用信息。膨胀层：通过卷积机器人的碰撞足迹，以指数衰减方式膨胀成本图中的致命障碍物。时空体素层：维护时间 3D 稀疏体积体素网格，该网格通过激光和 RGBD 相机的传感器模型随时间衰减。

D. Global Planner and Controller

全局规划器的任务是计算到达目标的最短路径，控制器使用局部信息来计算最佳局部路径和控制信号。全局规划器和控制器是委托给特定于任务的异步服务器的插件。为了性能，相关的全局和本地成本图位于各自的服务器中。

通过将 TEB 局部规划器与用于动态 3D 感知的 STVL 相结合，采用新算法在高度动态的环境中平稳导航。 The TEB controller is capable of taking in object detections and tracks to inform the controller of additional constraints on the environ- ment. It is also suitable for use on differential, omnidirectional,

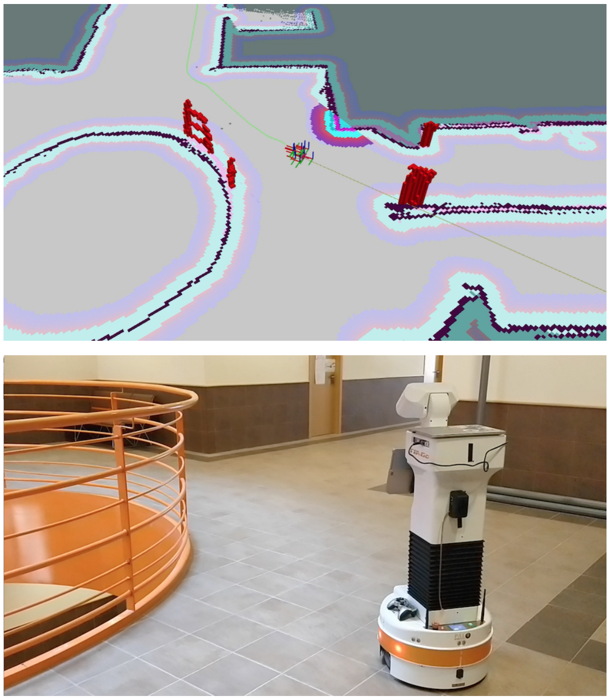

Fig. 4: Visualization of robot with depth, and map information.

and ackermann style robots. DWB, a DWA implementation (DW ”B” being incrementally named), is also implemented as a highly-configurable scoring-based controller. However it is not used in these experiments due to its performance in dynamic scenes. Those used for experiments are as follows:

A* Planner: Navigation function planner using A* expansion and assumes a 2D holonomic particle. **TEB Controller**: Uses Timed-elastic-bands for time-optimal point-to-point non-linear model predictive control.

E. State Estimation

For state estimation, Navigation2 follows ROS transformation tree standards to make use of many modern tools available from the community. This includes **Robot Localization**, a general sensor fusion solution using Extended or Unscented Kalman Filters [11]. It is used to provide smoothed base odometry from N arbitrary sources, which often includes wheel odometry, multiple IMUs, and visual odometry algorithms.

However, locally filtered poses are insufficient to account for integrated odometric drift, thus a global localization solution remains necessary for navigation. The global localization solutions used for experiments are as follows:

SLAM Toolbox: A configurable graph-based SLAM system using 2D pose graphs and canonical scan matching to generate a map and serialized files for multi-session mapping [24]. This was used to create the static map in preparation for the experiment. **AMCL**: An implementation of an Adaptive Monte-Carlo Localization, which uses a particle filter to localize a robot in a given occupancy grid using omni-directional or differential motion models [18]. This was used as the primary localization tool for the duration of the experiments.

F. Utilities

Navigation2 also includes tools to aid in testing and operations. It includes the *Lifecycle Manager* to coordinate the program lifecycle of the navigator and various servers. This manager will step each server through the managed node lifecycle: inactive, active, and finalized.

The lifecycle manager can be set to transition all provided servers to the active state on system bringup. Alternatively, it may wait for a signal to activate the system. Navigation2 enables users to activate, manage, and command their system through a GUI. This GUI includes controlling the lifecycle manager, issuing navigation commands, selecting waypoints to navigate through, and canceling current tasks in real-time. For autonomous applications, all of these commands are available through ROS2 server interfaces with examples.

G. Quality Assurance

System wide integration tests are employed for quality assurance purposes; helping to avoid regressions over time. In addition to basic unit testing, code style linters, and memory static analysis - simulation tests are used by a continuous integration pipeline to emulate an entire robot software stack. As shown in Figure 5, a 3D robotic simulator, Gazebo [25], is used to test community contributions over an assortment of virtual scenarios including static and dynamic environments.

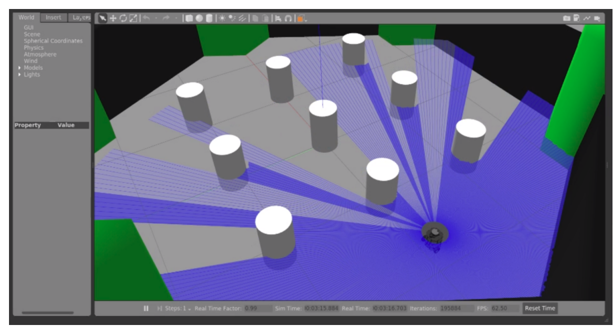

Fig. 5: Sample simulation from Continuous Integration tests.

Such system tests allow maintainers to monitor not only navigation failures over tested trajectories, but also performance analytics for various algorithms over an assortment of robot platforms with unique locomotion kinematics.

V. Experiment and Analysis

A. Experiment Overview

To show the robustness of Navigation2, we conducted a more extensive experiment than proposed in [1], an ”ultra-marathon” (a distance greater than a standard marathon). This test demonstrated the robustness and reliability of our system in long-term operations without human assistance using two professional robots: the Tiago and RB-1 base, shown in Figure 1. These robots operated in a human-filled environment in a University setting running at near-industrial speeds.

The experiment took place in the Technical School of Telecommunications Engineers at the Rey Juan Carlos University. Figure 7 shows the map of this environment with

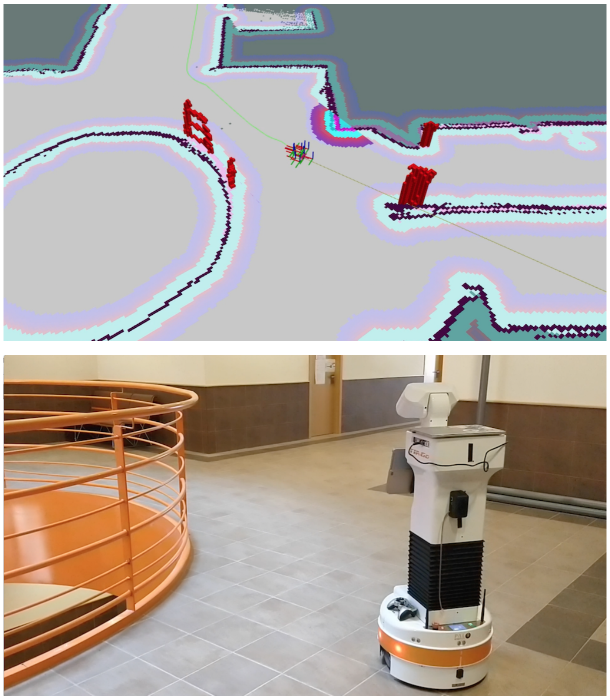

图4：Vi 具有深度和地图信息的机器人模型化 化。

和阿克曼风格的机器人。 DWB 是一种 DWA 实现（DW “B”逐渐命名），也被实现为高度可配置的基于评分的控制器。然而，由于其在动态场景中的性能，这些实验中并未使用它。实验所用的如下：

A* 规划器：使用 A* 展开的导航功能规划器并假设 2D 完整粒子。 TEB 控制器：使用定时弹性带进行时间最优点对点非线性模型预测控制。

E. *State Estimation*

对于状态估计，Navigation2 遵循 ROS 转换树标准，以利用社区提供的许多现代工具。这包括机器人定位，这是一种使用扩展或无味卡尔曼滤波器的通用传感器融合解决方案[11]。它用于提供来自 *N* 任意源的平滑基础里程计，通常包括车轮里程计、多个 IMU 和视觉里程计算法。

然而，局部过滤的姿态不足以解释积分里程漂移，因此导航仍然需要全局定位解决方案。实验使用的全局本地化方案如下：

SLAM Toolbox：一个可配置的基于图形的 SLAM 系统，使用 2D 位姿图和规范扫描匹配来生成地图和序列化文件以进行多会话映射 [24]。这用于创建静态地图，为实验做准备。 AMCL：自适应蒙特卡罗定位的一种实现，它使用粒子滤波器通过全向或差分运动模型将机器人定位在给定的占用网格中[18]。这被用作实验期间的主要定位工具。

F. *Utilities*

Navigation2 还包括帮助测试和操作的工具。它包括协调导航器和各种服务器的程序生命周期的*Lifecycle Manager*。该管理器将逐步执行每个服务器的受管节点生命周期：非活动、活动和最终确定。

可以将生命周期管理器设置为在系统启动时将所有提供的服务器转换为活动状态。或者，它可以等待信号来激活系统。 Navigation2 使用户能够通过 GUI 激活、管理和命令其系统。该 GUI 包括控制生命周期管理器、发出导航命令、选择要导航的航路点以及实时取消当前任务。对于自主应用程序，所有这些命令都可以通过 ROS2 服务器接口和示例获得。

G. *Quality Assurance*

采用系统范围的集成测试来保证质量；有助于避免随着时间的推移出现倒退。除了基本的单元测试、代码风格检查和内存静态分析之外，持续集成管道还使用模拟测试来模拟整个机器人软件堆栈。如图 5 所示，3D 机器人模拟器 Gazebo [25] 用于测试社区对各种虚拟场景（包括静态和动态环境）的贡献。

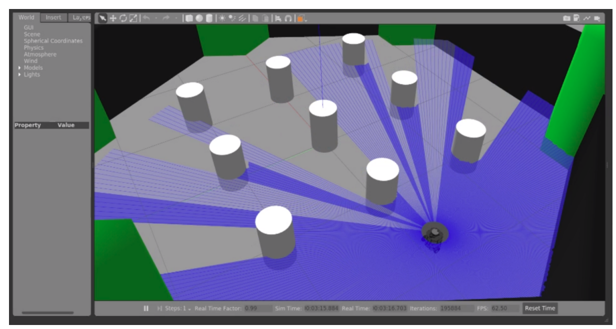

图 5：持续集成测试的示例模拟。

此类系统测试使维护人员不仅可以监控测试轨迹上的导航故障，还可以对具有独特运动学的各种机器人平台上的各种算法进行性能分析。

五、实验与分析

A. *Experiment Overview*

为了展示 Navigation2 的稳健性，我们进行了比[1]中提出的更广泛的实验，即“超级马拉松”（比标准马拉松更长的距离）。该测试证明了我们的系统在没有人工协助的情况下长期运行的稳健性和可靠性，使用了两个专业机器人：Tiago 和 RB-1 基地，如图 1 所示。这些机器人在大学环境中的人类环境中以接近工业速度运行。

该实验在胡安卡洛斯国王大学电信工程师技术学院进行。图 7 显示了该环境的地图

Fig. 6: Robots during the experiment.

the track used in the experiment. The robot was made to navigate this track around a central stairwell (surrounding waypoints 3, 11, 13, 14), a high-traffic bridge and hallway (waypoints 4-10), and back into a lab (waypoint 1) through a narrow doorway without intervention. The robot shared the environment alongside students, Figure 6, whom sometimes spontaneously block the path of the robot. We define a 300 meters route for the experiment through this set of waypoints.

TABLE II: Robot specifications.

	Tiago	**RB-1**
Robot Type	Base + Torso	Base
Dimensions	0.54 m x 1.45 m	0.50 m x 0.251 m
Max. Speed	1.0 m/s	1.5 m/s
Battery	2 x 36V 20Ah	24V 30Ah
Compute	Intel i7-7700	Intel i7-7567U
Memory	8GB Ram/250GB SSD	8GB Ram/120GB SSD
Operating System	Ubuntu 16.04	Ubuntu 18.04
Depth Camera	Orbbec Astra S	Orbbec Astra
Laser Sensor	Sick TIM561	Sick Tim571
Navigation2	External	Onboard

The characteristics of both robots are similar, described in Table II. Both use differential steering, with similar diameters but varying heights. They each use RGBD cameras and safety laser scanners with different ranges and resolutions for perception and obstacle avoidance. Both robots were set to have a maximum speed of 0.45 m/s for functional safety, below their maximum allowable speed. Two different robots were used to show the portability of our system across multiple hardware vendors with trivial reconfiguration. The Tiago utilized an external computer because its internal operating system is not supported by ROS2. Both robot vendors provide supported drivers and interfaces for users. We created a custom bridge between it and ROS2 which supports continuous flow of data. This was required to support high-frequency data requirements on transformations and odometry.

The robot saves the following information once a second:

- `timestamp`: Current time.
- `distance`: Odometric distance travelled.
- `recovery_executed`: Logged executed recoveries.
- `vel_x`: Instantaneous linear velocity.
- `vel_theta`: Instantaneous angular velocity.

All of the software, configurations, and data used in this experiment is available in the marathon[3] repository. This repository contains instructions to reproduce the experiment as well as the data described above collected in the experiment. We collected the following information for the experiment dataset: localized pose with covariance, instantaneous speed commanded, whether a recovery behavior was triggered, time and distance navigated, laser scan readings, geometric transformations, inertial sensor data, and map. This data is publicly available in the repository for experimental verification.

B. Analysis

TABLE III: Experiment results for both robots.

	RB1	**TIAGo**	**Total**
Time (hrs)	9.4	13.4	**22.8**
Distance (miles)	15.6	21.8	**37.4**
Recoveries	52	116	**168**
Recoveries per mile	3.3	5.3	**4.3**
Avg. speed (m/s)	0.39	0.35	**0.37**
Max speed (m/s)	0.45	0.45	**0.45**
Num. of Collisions	0	0	**0**
Num. of Emergency stops	0	0	**0**

Table III shows the results of the experiment. The robots successfully navigated over 37 miles in under 23 hours in a dynamic campus environment. During the experiment the average linear speed was 0.37 m/s. Neither robot suffered a collision or a dangerous situation requiring an emergency stop during the test. The robots operated without direct assistance throughout the experiment and never failed to complete a navigation task.

Figure 8 shows a time-lapse panel of a situation that the robot faced in a crowded hallway during the moments of the start of classes. In this period, many students are walking quickly to their destinations, frequently using phones. This sequence shows how the robot interacts with several groups of students. On approach, the robot first slows and navigates around first two people in the group in the first panel. The third person pauses, giving the robot the right of way to continue – in panel 2. Then, after navigating around the first group in panels 1-3, a new person from panel 3 to 4 walks through the scene and the robot corrects its trajectory to navigate away from their path. Panel 5 shows the robot exiting the complex scenario without collision or stoppage.

Passive assistance was rendered to the robot on rare instances when students occupied the exact pose of a waypoint the robot was navigating towards. At that time, students were asked to move to allow the robot to continue. The time impact of this passive assistance was under 1 minute over the 22.8 hour experiment. This was a fault in the application built for the marathon experiment, which utilizes Navigation2. The

[3]https://github.com/IntelligentRoboticsLabs/marathon_ros2

图 6：实验中的机器人。

实验中使用的轨道。机器人沿着这条轨道绕过中央楼梯间（围绕路点 3、11、13、14）、交通繁忙的桥梁和走廊（路点 4-10），并在无人干预的情况下通过狭窄的门口返回实验室（路点 1）。机器人与学生共享环境（图 6），学生有时会自发地挡住机器人的路径。我们通过这组路点定义了一条 300 米的实验路线。

表 II：机器人规格。

	Tiago	RB-1
Robot Type	Base + Torso	Base
Dimensions	0.54 m x 1.45 m	0.50 m x 0.251 m
Max. Speed	1.0 m/s	1.5 m/s
Battery	2 x 36V 20Ah	24V 30Ah
Compute	Intel i7-7700	Intel i7-7567U
Memory	8GB Ram/250GB SSD	8GB Ram/120GB SSD
Operating System	Ubuntu 16.04	Ubuntu 18.04
Depth Camera	Orbbec Astra S	Orbbec Astra
Laser Sensor	Sick TIM561	Sick Tim571
Navigation2	External	Onboard

两种机器人的特征相似，如表二所示。两者都使用差速转向，直径相似但高度不同。它们各自使用具有不同范围和分辨率的 RGBD 摄像机和安全激光扫描仪来进行感知和避障。出于功能安全考虑，两台机器人的最大速度均设置为 0.45 m/s，低于其最大允许速度。使用两个不同的机器人来展示我们的系统通过简单的重新配置在多个硬件供应商之间的可移植性。 Tiago 使用外部计算机，因为其内部操作系统不受 ROS2 支持。两家机器人供应商都为用户提供支持的驱动程序和接口。我们在它和 ROS2 之间创建了一个自定义桥梁，支持连续的数据流。这是为了支持转换和里程计的高频数据需求。

机器人每秒保存一次以下信息：

- 时间戳：当前时间。
- 距离：行驶的里程距离。
- recovery_execused：记录执行的恢复。
- vel_x：瞬时线速度。
- vel_theta：瞬时角速度。

本实验中使用的所有软件、配置和数据都可以在 marathon[3] 存储库中找到。该存储库包含重现实验的说明以及实验中收集的上述数据。我们为实验数据集收集了以下信息：具有协方差的局部姿态、指令的瞬时速度、是否触发恢复行为、导航的时间和距离、激光扫描读数、几何变换、惯性传感器数据和地图。该数据可在存储库中公开获取，以供实验验证。

B. Analysis

表 III：两个机器人的实验结果。

	RB1	TIAGo	Total
Time (hrs)	9.4	13.4	**22.8**
Distance (miles)	15.6	21.8	**37.4**
Recoveries	52	116	**168**
Recoveries per mile	3.3	5.3	**4.3**
Avg. speed (m/s)	0.39	0.35	**0.37**
Max speed (m/s)	0.45	0.45	**0.45**
Num. of Collisions	0	0	**0**
Num. of Emergency stops	0	0	**0**

表III显示了实验结果。机器人在动态的校园环境中在 23 小时内成功导航超过 37 英里。实验期间平均线速度为0.37 m/s。测试过程中，两个机器人都没有发生碰撞或需要紧急停止的危险情况。在整个实验过程中，机器人在没有直接帮助的情况下进行操作，并且从未未能完成导航任务。

图 8 显示了机器人在上课时在拥挤的走廊中所面临的情况的延时面板。在此期间，许多学生快速步行前往目的地，并频繁使用手机。该序列显示了机器人如何与多组学生互动。在接近时，机器人首先减慢速度并绕过第一个面板中小组中的前两个人。第三个人暂停，给机器人让路继续——在面板 2 中。然后，在面板 1-3 中绕过第一组后，面板 3 到 4 中的一个新人走过场景，机器人纠正其轨迹以远离他们的路径。图 5 显示机器人在没有碰撞或停止的情况下退出复杂场景。

在极少数情况下，当学生占据机器人导航的航路点的精确姿势时，机器人会获得被动帮助。当时，学生们被要求移动，让机器人继续前进。在 22.8 小时的实验中，这种被动辅助的时间影响不到 1 分钟。这是为马拉松实验构建的应用程序中的一个错误，该应用程序利用了 Navigation2。这

3https://github.com/IntelligentRoboticsLabs/marathon_ros2

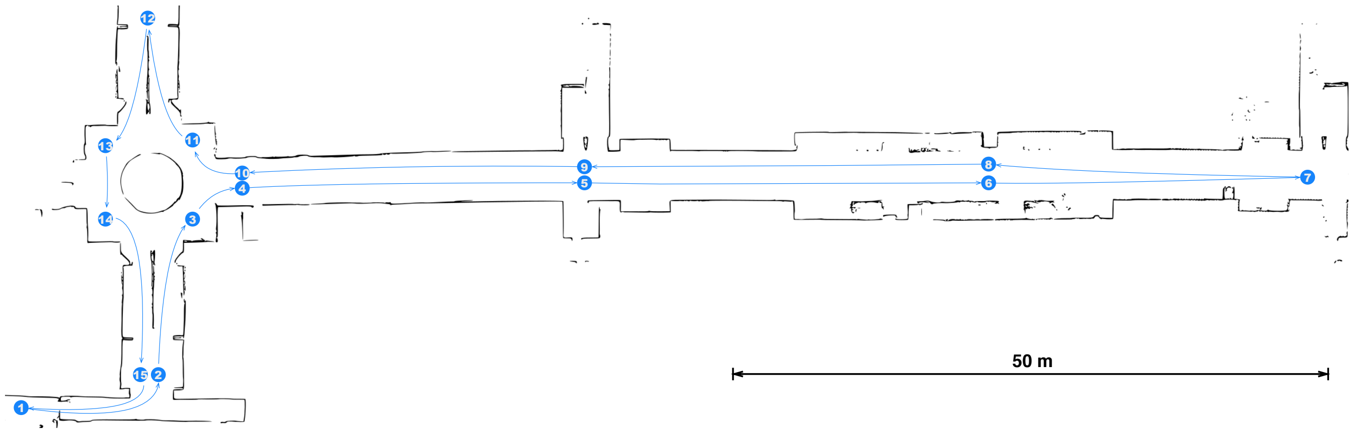

Fig. 7: Map of hallway, stairwell, and lab used in experiments.

Fig. 8: An example robot-pedestrian interaction.

application did not account for occupation of the goal pose, however, this does not represent an issue or failure mode of Navigation2 itself.

While the total number of recoveries seems high, triggering recovery behaviors is not generally a poor-performance indicator since it is part of a fault-tolerant system. Recoveries indicated that the robot had some difficulty due to a potentially blocked path or erroneous sensor measurement. The system automatically performs the recovery required to overcome a given challenge resulting in the robust system presented.

The majority of recovery behaviors executed were due to two cases. The first happened in crowded spaces where the robot was unable to compute a route to its goal. In this case, the robot cleared its environmental representation, usually the local costmap, of obstacles. If this was insufficient, typically, the path was blocked by students and the wait recovery paused navigation until the path was clear. The second case was when the localization confidence was low. Long corridor areas with repetitive features in the presence of many people around caused occasional issues. AMCL works well in dynamic environments when enough rays can reach the static map, however, many changes over a long duration can degrade positioning quality. In this situation, the robot's spin recovery helped regain localization confidence to continue on its path.

A video highlight reel of this experiment can be found in the footnote [4].

[4]https://www.youtube.com/watch?v=tXp8wFZr68M

VI. Limitations

While shown in experiments to effectively navigate in narrow spaces around people for many miles without human intervention, there are some limitations to the Navigation2 system which will be addressed in future work. Currently, the A* planner used does not create feasible paths for non-circular non-holonomic robots. This does not generally cause an issue for differential drive robots used in this experiment as they can rotate to a new heading in-place. However, this creates an issue for complex geometry robots or car-like robots resulting in an inability to move because of an infeasible path to follow. Extensions are in development to support non-holonomic robots of arbitrary shape.

Further, the TEB controller can account for dynamic obstacles in computing velocity commands, however, no explicit obstacle detection was utilized in these experiments. Without these explicit detections, our system was able to successfully navigate in the presence of many dynamic obstacles. However, detections and predictive models will further enable robots to operate safely around humans and other agents.

VII. Conclusion

It has been shown that the Navigation2 navigation system can reliably navigate a campus environment in the presence of students during long-duration testing. Our demonstration had 2 different industrial-grade robots, using our system, navigate in excess of a marathon without any human intervention or collision. The system made use of behavior trees to orchestrate navigation algorithms to be highly configurable and leverage

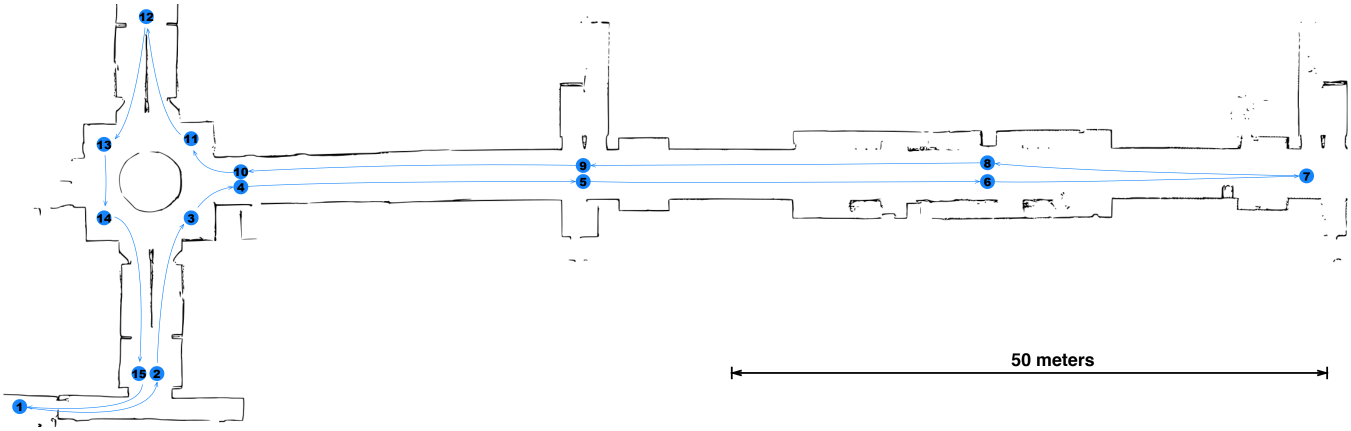

图 7：实验中使用的走廊、楼梯间和实验室的地图。

图 8：机器人与行人交互的示例。

应用程序没有考虑目标姿势的占用，但这并不代表 Navigation2 本身的问题或故障模式。

虽然恢复总数看起来很高，但触发恢复行为通常并不是性能不佳的指标，因为它是容错系统的一部分。恢复结果表明，由于路径可能被堵塞或传感器测量错误，机器人遇到了一些困难。系统自动执行克服给定挑战所需的恢复，从而形成强大的系统。

大多数执行的恢复行为是由于两种情况。第一次发生在拥挤的空间，机器人无法计算到达目标的路线。在这种情况下，机器人清除了其环境表示（通常是本地成本地图）的障碍物。如果这还不够，通常情况下，路径会被学生挡住，等待恢复会暂停导航，直到路径畅通为止。第二种情况是本地化信心较低时。在周围有很多人的情况下，具有重复特征的长走廊区域偶尔会引起问题。当足够的光线可以到达静态地图时，AMCL 在动态环境中效果很好，但是，长时间内的许多变化可能会降低定位质量。在这种情况下，机器人的旋转恢复有助于重新获得定位信心，以继续其路径。

该实验的视频集锦可以在脚注 [4] 中找到。

[4]https://www.youtube.com/watch?v=tXp8wFZr68M

六．局限性

虽然实验表明，Navigation2 系统可以在没有人为干预的情况下在人们周围的狭窄空间中有效导航数英里，但仍存在一些限制，这些限制将在未来的工作中得到解决。目前，使用的 A* 规划器无法为非圆形非完整机器人创建可行路径。对于本实验中使用的差动驱动机器人来说，这通常不会造成问题，因为它们可以就地旋转到新的航向。然而，这给复杂几何机器人或类汽车机器人带来了一个问题，导致由于遵循不可行的路径而无法移动。正在开发扩展以支持任意形状的非完整机器人。

此外，TEB 控制器可以在计算速度命令时考虑动态障碍，但是，在这些实验中没有使用明确的障碍物检测。如果没有这些明确的检测，我们的系统就能够在存在许多动态障碍物的情况下成功导航。然而，检测和预测模型将进一步使机器人能够在人类和其他代理周围安全运行。

七．结论

事实证明，在长时间测试期间，Navigation2 导航系统可以在有学生在场的情况下可靠地在校园环境中导航。我们的演示有 2 个不同的工业级机器人，使用我们的系统，在没有任何人为干预或碰撞的情况下完成马拉松比赛。该系统利用行为树来编排导航算法，使其具有高度可配置性和利用性

multi-core processors – enabled by ROS2's industrial-grade reliable communications. We used modern algorithms such as STVL and TEB for perception and control in large and dynamic environments while continuing to build on the legacy of the popular work of the ROS Navigation Stack.

There are gaps in dynamic obstacle tracking and planning which are being developed and will be addressed in future work. This work is open-source under permissive licensing and we anticipate many more extensions and algorithms to be created using our framework, further improving over time.

ACKNOWLEDGEMENTS

Other contributors on *Navigation2* include: Matt Hansen, Carl Delsey, Brian Wilcox, Mohammad Haghighipanah, Mike Jeronimo, and Carlos Orduno. Authors also thanks Lorena Bajo for her work during experiments.

REFERENCES

[1] Marder-Eppstein, E. Berger, T. Foote, B. Gerkey, and K. Konolige, "The Office Marathon: Robust navigation in an indoor office environment," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), May 2010, pp. 300307.

[2] W. Burgard, A. Cremers, D. Fox, D. Hahnel, G. Lakemeyer, D. Schulz, W. Steiner, and S. Thrun, "The interactive museum tour-guide robot," in Proc. of the National Conference on Artificial Intelligence, 1998.

[3] S.F. Chik, Y. Fai, E. Su, T.Y Lim, Y. Subramaniam, P.J.H. Chin, "A review of social-aware navigation frameworks for service robot in dynamic human environments", Journal of Telecommunication, Electronic and Computer Engineering, 8. 41-50, 2016.

[4] K. Konolige, "A gradient method for realtime robot control," in Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2000), Kagawa University, Takamatsu, Japan, 2000.

[5] D. Fox, W. Burgard and S. Thrun, "The dynamic window approach to collision avoidance," in IEEE Robotics and Automation Magazine, vol. 4, no. 1, pp. 23-33, March 1997.

[6] M. Colledanchise, P. Ogren, Use of BTs in Robotics and AI, in Behavior Trees in Robotics and AI: An Introduction, 1st Ed., CRC Press, 2018, pp. 15-22.

[7] Y. Maruyama, S. Kato, T. Azumi, "Exploring the Performance of ROS2", Proc. of ACM EMSOFT, pp. 5:1-5:10, 2016.

[8] S. Macenski, D. Tsao, M. Feinberg, "Spatio-Temporal Voxel Layer: A View on Robot Perception for the Dynamic World," International Journal of Advanced Robotic Systems, 2020, to appear.

[9] D. V. Lu, D. Hershberger, W. D. Smart, "Layered costmaps for context-sensitive navigation", 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2014), pp. 709-715.

[10] C. Rsmann, F. Hoffmann, T. Bertram, "Timed-elastic-bands for time-optimal point-to-point nonlinear model predictive control", European Control Conference (ECC), 2015.

[11] T. Moore and D. Stouch. A generalized extended kalman filterimplementation for the robot operating system. InIntelligentAutonomous Systems 13, pages 335348. Springer, 2016.

[12] Pal Robotics, Tiago, http://pal-robotics.com/robots/tiago/. Accessed On: February 5, 2020.

[13] Robotnik Robotics, RB-1, https://www.robotnik.eu/manipulators/rb-one/. Accessed On: February 5, 2020.

[14] I. Nourbakhsh, R. Powers, and S. Birchfield, DERVISH An Office-Navigating Robot, AIMag, vol. 16, no. 2, p. 53, Jun. 1995.

[15] S. Thrun, M. Bennewitz, W. Burgard, A. B. Cremers, F. Dellaert, D. Fox, D. Hhnel, C. Rosenberg, N. Roy, J. Schulte, and D. Schulz, "Minerva: A second-generation museum tour-guide robot, in In Proceedings of IEEE International Conference on Robotics and Automation (ICRA99), 1999.

[16] R. Simmons and S. Koenig. "Probabilistic robot navigation in partially observable environments". In Proceedings of the 14th international joint conference on Artificial intelligence - Volume 2 (IJCAI95). 1995. San Francisco, CA, USA, pp. 10801087.

[17] Astrom, K.J. "Optimal control of Markov processes with incomplete state information". Journal of Mathematical Analysis and Applications. 1965. 10: pp. 174205.

[18] D. Fox, "KLD-Sampling: Adaptive Particle Filters", Advances in Neural Information Processing Systems, 14, pp. 713-720. 2002.

[19] R. Kuemmerle, M. Ruhnke, B. Steder, C. Stachniss, and W. Burgard, A Navigation System for Robots Operating in Crowded Urban Environments, in ICRA, 2013.

[20] Rahib H. Abiyev, Irfan Gnsel, Nurullah Akkaya, Ersin Aytac, Ahmet aman, Sanan Abizada, "Robot Soccer Control Using Behaviour Trees and Fuzzy Logic", 12th International Conference on Application of Fuzzy Systems and Soft Computing ICAFS 2016 Book Series: Procedia Computer Science, vol. 102, pp. 477-484, 2016.

[21] G. Biggs, T. Foote, "Managed Nodes", https://design.ros2.org/articles/node_lifecycle.html. Accessed On: February 12, 2020.

[22] J.Huang and T. Lau, "Design and Evaluation of a Rapid Programming System for Service Robots. Proceedings of the 11th ACM/IEEE International Conference on Human Robot Interaction. (2016), 295302.

[23] Kone, Service Robot API, https://developer.kone.com/s/news/service-robot-api-20Y2p000000CljREAS. Accessed on February 11, 2020.

[24] GitHub, "SLAM Toolbox", https://github.com/SteveMacenski/slam_toolbox. Accessed on February 27, 2020.

[25] N. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566), Sendai, 2004, pp. 2149-2154 vol.3.

多核处理器——由 ROS2 的工业级可靠通信支持。我们使用 STVL 和 TEB 等现代算法在大型动态环境中进行感知和控制，同时继续在 ROS 导航堆栈的流行工作的基础上进行构建。

动态障碍物跟踪和规划方面存在差距，这些差距正在开发中，并将在未来的工作中得到解决。这项工作是在许可许可下开源的，我们预计将使用我们的框架创建更多的扩展和算法，并随着时间的推移进一步改进。

致谢

Navigation2 的其他贡献者包括：Matt Hansen、Carl Delsey、Brian Wilcox、Mohammad Haghipanah、Mike Jeronimo 和 Carlos Orduno。作者还感谢 Lorena Bajo 在实验期间所做的工作。

参考

[1] Marder-Eppstein、E. Berger、T. Foote、B. Gerkey 和 K. Konolige，“办公室马拉松：室内办公环境中的稳健导航”，载于 Proc。 IEEE 国际。会议。机器人。自动。 (ICRA)，2010 年 5 月，第 300307 页。 [2] W. Burgard、A. Cremers、D. Fox、D. Hahnel、G. Lakemeyer、D. Schulz、W. Steiner 和 S. Thrun，“交互式博物馆导游机器人”，载于 Proc. (ICRA)，第 300307 页。全国人工智能会议，1998。 [3] S.F. Chik、Y. Fai、E. Su、T.Y Lim、Y. Subramaniam、P.J.H. Chin，“动态人类环境中服务机器人社交感知导航框架的综述”，电信、电子与计算机工程杂志，8. 41-50，2016。 [4] K. Konolige，“实时机器人控制的梯度方法”，IEEE/RSJ 国际智能机器人和系统会议记录 (IROS 2000)，香川大学，高松，日本，2000. [5] D. Fox、W. Burgard 和 S. Thrun，“避免碰撞的动态窗口方法”，载于 IEEE 机器人与自动化杂志，第 1 卷。 4、没有。 1，第 23-33 页，1997 年 3 月。 [6] M. Colledanchise、P. Ogren，BT 在机器人和人工智能中的使用，《机器人和人工智能中的行为树：简介》，第一版，CRC Press，2018 年，第 15-22 页。 [7] Y. Maruyama、S. Kato、T. Azumi，“探索 ROS2 的性能”，Proc。 ACM EMSOFT，第 5:1-5:10，2016 年。 [8] S. Macenski、D. Tsao、M. Feinberg，“时空体素层：动态世界的机器人感知视图”，国际先进机器人系统杂志，2020 年，即将发表。 [9] D. V. Lu、D. Hershberger、W. D. Smart，“上下文敏感导航的分层成本图”，2014 年 IEEE/RSJ 国际智能机器人和系统会议 (IROS 2014)，第 709-715 页。 [10] C. Rsmann、F. Hoffmann、T. Bertram，“时间最优点对点非线性模型预测控制的定时弹性带”，欧洲控制会议 (ECC)，2015 年。 [11] T. Moore 和 D. Stouch。机器人操作系统的广义扩展卡尔曼滤波器实现。 InIntelligentAutonomous Systems 13，第 335348 页。Springer，2016。[12] Pal Robotics，Tiago，http://pal-robotics.com/robots/tiago/。访问日期：2020 年 2 月 5 日。[13] Robotnik Robotics，RB-1，https://www.robotnik.eu/manipulators/rb-one/。访问日期：2020 年 2 月 5 日。 [14] I. Nourbakhsh、R. Powers 和 S. Birchfield，DERVISH 办公室导航机器人，AIMag，卷。 16、没有。 2，第 14 页。 53，1995 年 6 月。 [15] S. Thrun、M. Bennewitz、W. Burgard、A. B. Cremers、F. Dellaert、D. Fox、D. Hhnel、C. Rosenberg、N. Roy、J. Schulte 和 D. Schulz，“Minerva：第二代博物馆导游机器人，载于 IEEE 国际机器人与自动化会议论文集(ICRA99)，1999。 [16] R. Simmons 和 S. Koenig。第 14 届国际人工智能联合会议论文集 - 第 2 卷 (IJCAI95)。1995 年，美国加利福尼亚州旧金山，第 10801087 页。“具有不完整状态信息的马尔可夫过程的最优控制”。数学分析与应用杂志，1965 年。10：第 174205 页。

[18] D. Fox，“KLD 采样：自适应粒子滤波器”，神经信息处理系统进展，14，第 713-720 页。 2002. [19] R. Kuemmerle、M. Ruhnke、B. Steder、C. Stachniss 和 W. Burgard，在拥挤城市环境中运行的机器人导航系统，ICRA，2013 年。 [20] Rahib H. Abiyev、Irfan Gnsel、Nurullah Akkaya、Ersin Aytac、Ahmet aman、Sanan Abizada，“使用行为树和模糊逻辑的机器人足球控制”，第十二届模糊系统和软计算应用国际会议 ICAFS 2016 丛书：Procedia 计算机科学，卷。 102，第 477-484 页，2016 年。 [21] G. Biggs、T. Foote，“托管节点”，https://design.ros2.org/articles/nodelifecycle.html。访问日期：2020 年 2 月 12 日。 [22] J.Huang 和 T. Lau，“服务机器人快速编程系统的设计与评估。第 11 届 ACM/IEEE 国际人类机器人交互会议论文集。(2016)，295302。[23] Kone，服务机器人 API，https://developer.kone.com/s/news/ service-robot-api-20Y2p000000CljREAS。2020 年 2 月 11 日访问。[24] GitHub，“SLAM Toolbox”，https://github.com/SteveMacenski/slam toolbox。2020 年 2 月 27 日访问。[25] N. Koenig 和 A. Howard，“开源多机器人 Gazebo 的设计和使用范例”模拟器”，2004 年 IEEE/RSJ 国际智能机器人和系统会议 (IROS)（IEEE Cat.No.04CH37566），仙台，2004 年，第 2149-2154 页，第 3 卷。